

Auditor Léxico de Telemetria de Sensores Industriais

Um firewall léxico construído com Expressões Regulares e AFNε

Linguagens Formais e Autômatos · Trabalho do 1º Bimestre · CESUPA

Augusto Rodrigues · Caue Jadão · César Augusto

github.com/cauebarroso/Auditor-de-sensores-industriais-com-express-es-regulares

O problema e a solução

- Sensores e atuadores conversam com o supervisor (SCADA/IloT) por pacotes de texto.
- Um único símbolo errado vira uma leitura falsa ou uma ordem perigosa.
- Solução: um firewall léxico. O pacote só passa se pertencer à linguagem de uma das cinco ERs.
- Pacotes bloqueados recebem a causa provável e a coluna exata do erro, obtida simulando o AFNε.

Pacotes que chegam à rede

SEN-TMP-0101:TEMP=78.4C	✓
CMD ATU-VLV-0201 AJUSTAR 40%	✓
10.20.0.0/16	✓
2026-09-30T06:42:10 WARN SEN-TMP-0101: ...	
SEN-TMP-0101:TEMP=104.7C	X
letra O no lugar do zero	
CMD ATU-VLV-0201 AJUSTAR 120%	X
ajuste acima de 100%	
2026-09-31T07:10:00 INFO ...	X
setembro tem 30 dias	

Entradas → processamento → saídas

Entradas

- Pacote digitado (modo interativo)
- Pacote por argumento (--pacote)
- Arquivo .txt em lote, com comentários #

Processamento

- Normaliza (remove espaços das pontas)
- re.fullmatch nas 5 ERs
- Extrai campos dos válidos
- Diagnostica os inválidos pelo AFNε

Saídas

- [OK] / [ERRO] com coluna do erro
- Relatório de conformidade
- Filtros por categoria
- Exportação JSON, CSV ou TXT

Módulo	Responsabilidade
expressoes.py	As 5 ERs e as fichas (alfabeto, linguagem, ER formal)
afn.py · construtor_afn.py	AFNε: fecho-ε, simulação, construção de Thompson, prova de equivalência
diagnostico.py · relatorio.py · auditor.py	Diagnóstico, relatório/exportação e interface (menu e linha de comando)

Notação formal × sintaxe no código

Notação formal	No Python	Significado
$r s$	<code>rs</code>	Concatenação
$r \mid s$	<code>r s</code>	União
r^*	<code>r*</code>	Fecho de Kleene (zero ou mais)
$r r^*$	<code>r+</code>	Fecho positivo (uma ou mais)
$(r \mid \epsilon)$	<code>r?</code>	Opcionalidade
$r r r r$	<code>r{4}</code>	Repetição exata
$r r (r \mid \epsilon)$	<code>r{2,3}</code>	Repetição limitada = <code>rr</code> <code>rrr</code>
<code>[0-9]</code>	<code>[0-9]</code>	Classe finita: <code>(0 1 ... 9)</code>
<code>_ e .</code>	<code>' ' e \.</code>	Espaço e ponto literais (o ponto do Python precisa de escape)
<code><NOME> = r</code>	<code>—</code>	Definição regular: só um nome para a subexpressão

Sempre `re.fullmatch` (dispensa `^` e `$`). Proibidos e rejeitados pelo nosso analisador: `\d \w \s`, ponto curinga, classes negadas, retroreferências e lookarounds.

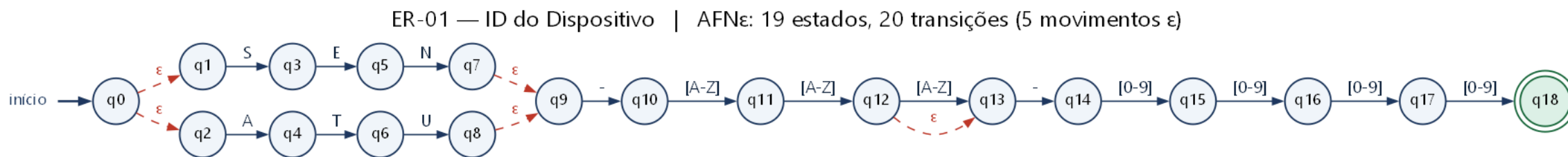
Identificador de um sensor (SEN) ou atuador (ATU); bloco reutilizado nas ERs 02, 03 e 05.

ER formal

```
<ID> = (SEN | ATU) - [A-Z][A-Z]([A-Z] | ε) - [0-9][0-9][0-9][0-9]
ER01 = <ID>
```

No código (re.fullmatch)

```
r"(SEN|ATU)-[A-Z]{2,3}-[0-9]{4}"
```



✓ SEN-TM-0001
✓ ATU-VLV-0420

✗ sen-tm-0001 ← minúsculas não pertencem a Σ_1
✗ SEN-TM-00012 ← 5 dígitos: {4} exige exatamente 4

Leitura de um sensor: temperatura (C), umidade (%) ou pressão (hPa), dentro da faixa.

ER formal

```
<SID> = SEN - [A-Z][A-Z]([A-Z] | ε) - [0-9][0-9][0-9][0-9]
<DEC> = . [0-9] | ε
<TEMP> = TEMP= (- | ε) (1[0-9][0-9] | ([1-9] | ε) [0-9]) <DEC> C
<UMID> = UMID= (100 (.0 | ε) | ([1-9] | ε) [0-9] <DEC>) %
<PRES> = PRES= (8[5-9][0-9] | 9[0-9][0-9] | 10[0-9][0-9]) hPa
ER02 = <SID> : (<TEMP> | <UMID> | <PRES>)
```

No código (re.fullmatch)

```
r"SEN-[A-Z]{2,3}-[0-9]{4}:(TEMP=-?(1[0-9][0-9]|[1-9]?[0-9])(\.[0-9])?C|UMID=(100(\.0)?|[1-9]?[0-9](\.[0-9])?)%|PRES=(8[5-9][0-9]|9[0-9][0-9]|10[0-9][0-9])hPa)"
```

Exemplos (tests/casos_teste.py)

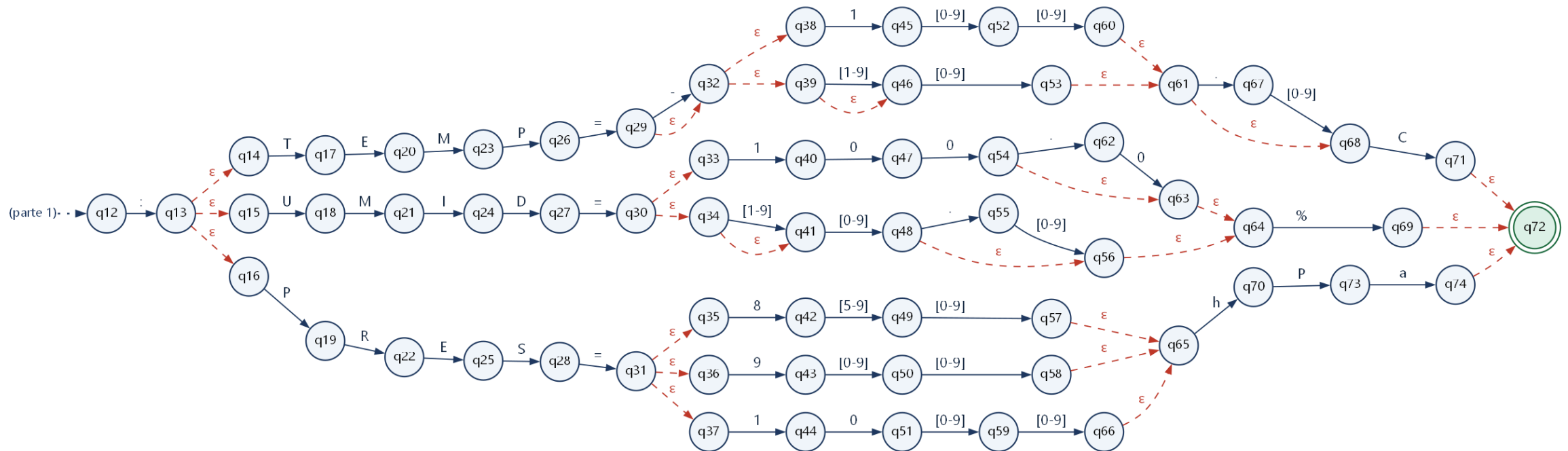
```
✓ SEN-TM-0001:TEMP=25.5C
✓ SEN-TM-0001:TEMP=-10C
✓ SEN-TMP-0002:TEMP=-199.9C
✗ SEN-TM-0001:TEMP=200C ← 200 > 199: fora da faixa
✗ SEN-TM-0001:UMID=100.5% ← 100.5% > 100%: fora da faixa
✗ SEN-TM-0001:PRES=849hPa ← 849 < 850: fora da faixa
```

ER-02 — AFN ϵ : o ID do sensor e a união das três medições

ER-02 — parte 1 de 2



ER-02 — parte 2 de 2



Ordem para um atuador: ligar, desligar, abrir, fechar ou ajustar de 0 a 100%.

ER formal

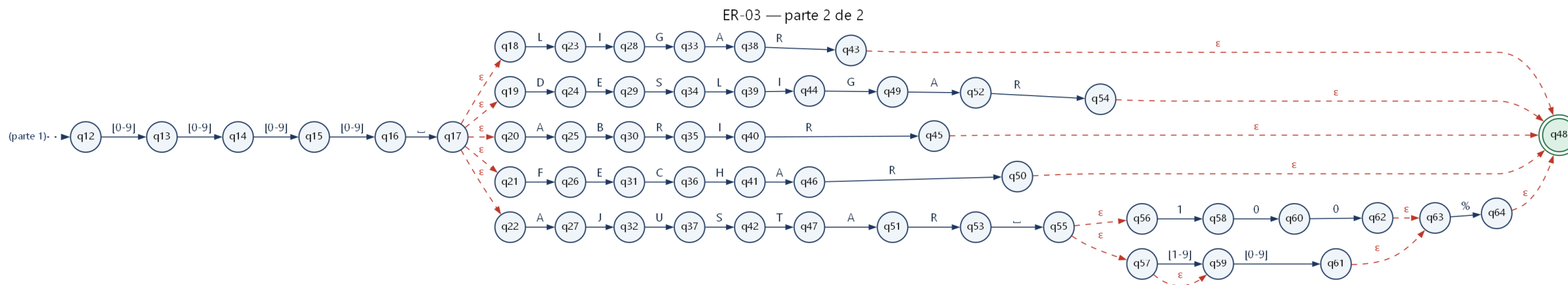
```

<AID> = ATU - [A-Z][A-Z]([A-Z] | ε) - [0-9][0-9][0-9][0-9]
<PCT> = 100 | ([1-9] | ε) [0-9]
ER03 = CMD _ <AID> _ (LIGAR | DESLIGAR | ABRIR | FECHAR | AJUSTAR _ <PCT> %)
    
```

No código (re.fullmatch)

```

r"CMD ATU-[A-Z]{2,3}-[0-9]{4} (LIGAR|DESLIGAR|ABRIR|FECHAR|AJUSTAR (100|[1-9]?[0-9]))%"
    
```



✓ CMD ATU-VLV-0420 LIGAR
 ✓ CMD ATU-VLV-0420 DESLIGAR

✗ CMD ATU-VLV-0420 AJUSTAR 101% ← 101 > 100: fora da faixa
 ✗ CMD SEN-VLV-0420 LIGAR ← sensor (SEN) não recebe comandos

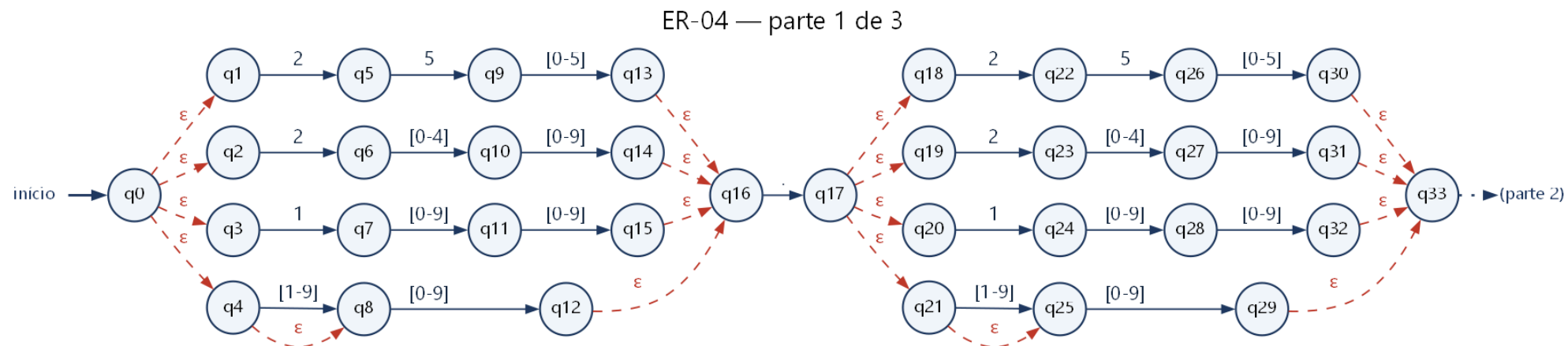
Endereço IPv4 com octetos de 0 a 255, sem zero à esquerda, e máscara CIDR opcional (/0 a /32).

ER formal

```
<OCT> = 25[0-5] | 2[0-4][0-9] | 1[0-9][0-9] | ([1-9] | ε) [0-9]
<MASC> = 3[0-2] | ([12] | ε) [0-9]
ER04 = <OCT> . <OCT> . <OCT> . <OCT> (/ <MASC> | ε)
```

No código (re.fullmatch)

```
r"((25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])\\.){3}(25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])(/(3[0-2]|([12]?[0-9]))?)?"
```



✓ 192.168.0.1
✓ 10.0.0.1/8

✗ 256.0.0.1 ← 256 > 255: octeto fora da faixa
✗ 192.168.0.1/33 ← máscara 33 > 32

Registro de evento: data válida, hora, severidade, dispositivo e mensagem.

ER formal

```
⟨DATA⟩ = [0-9][0-9][0-9][0-9] - ((0[1-9] | 1[0-2]) - (0[1-9] | [12][0-9]) | (0[13-9] | 1[0-2]) - 30 | (0[13578] | 1[02]) - 31)
⟨HORA⟩ = ([01][0-9] | 2[0-3]) : [0-5][0-9] : [0-5][0-9]
⟨SEV⟩ = INFO | WARN | ERROR | CRIT
⟨ID⟩ = (SEN | ATU) - [A-Z][A-Z]([A-Z] | ε) - [0-9][0-9][0-9][0-9]
⟨PAL⟩ = [A-Za-z0-9_.-] [A-Za-z0-9_.-]*
⟨MSG⟩ = ⟨PAL⟩ ( _ ⟨PAL⟩ ) *
ER05 = ⟨DATA⟩ T ⟨HORA⟩ _ ⟨SEV⟩ _ ⟨ID⟩ : _ ⟨MSG⟩
```

No código (re.fullmatch)

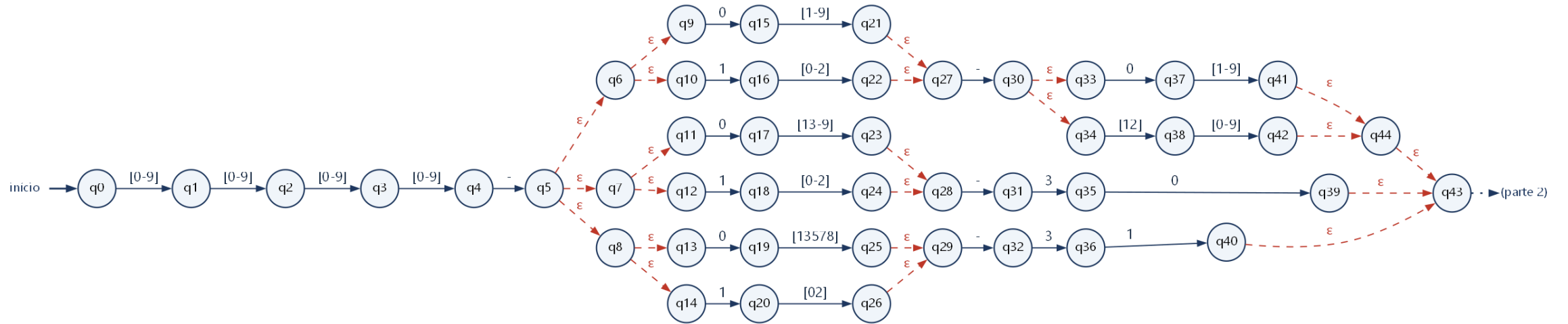
```
r"[0-9]{4}-((0[1-9]|1[0-2])-(0[1-9]|[12][0-9])|(0[13-9]|1[0-2])-30|(0[13578]|1[02])-31)T([01][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9](INFO|WARN|ERROR|CRIT) (SEN|ATU)-[A-Z]{2,3}-[0-9]{4}: [A-Za-z0-9_.-]+( [A-Za-z0-9_.-]+)*"
```

Exemplos (tests/casos_teste.py)

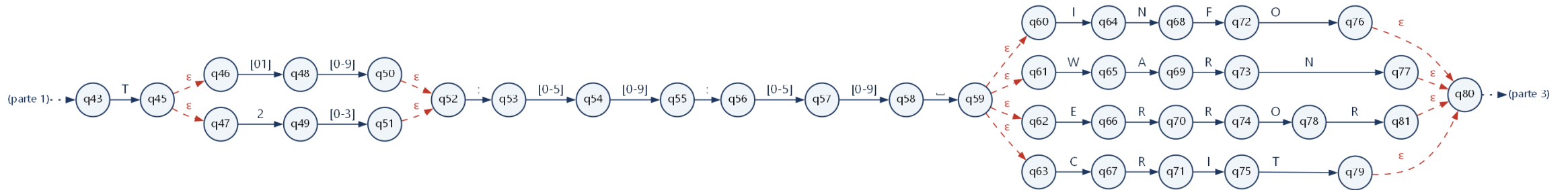
```
✓ 2026-09-30T08:15:00 INFO SEN-TM-0001: leitura normal
✓ 2026-12-31T23:59:59 CRIT ATU-XX-9999: falha critica no motor
✓ 2026-01-01T00:00:00 WARN SEN-AB-1234: bateria fraca
✗ 2026-13-01T10:00:00 INFO SEN-TM-0001: ok ← mês 13 não existe
✗ 2026-02-30T10:00:00 INFO SEN-TM-0001: ok ← 30 de fevereiro não existe
✗ 2026-04-31T10:00:00 INFO SEN-TM-0001: ok ← 31 de abril não existe
```

ER-05 — AFN ϵ da data e da hora

ER-05 — parte 1 de 4



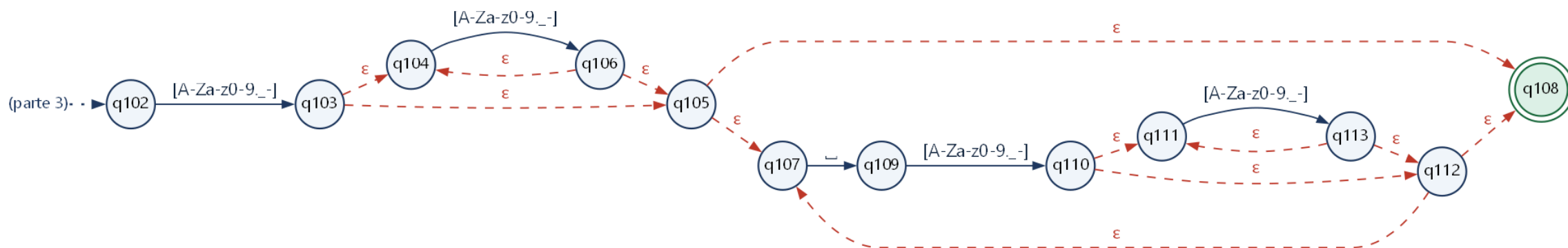
ER-05 — parte 2 de 4



ER-05 — a mensagem usa fecho de Kleene

$\langle \text{PAL} \rangle = [\text{A-Za-z0-9_.-}] [\text{A-Za-z0-9_.-}]^*$ $\langle \text{MSG} \rangle = \langle \text{PAL} \rangle (_ \langle \text{PAL} \rangle)^*$
No código: $[\text{A-Za-z0-9_.-}]^+ ([\text{A-Za-z0-9_.-}]^+)^*$

ER-05 — parte 4 de 4



- Laço interno: o fecho de $[\text{A-Za-z0-9_.-}]^*$ ($q104 \leftrightarrow q106$). Laço externo: $(_ \langle \text{PAL} \rangle)^*$, que volta de $q112$ para $q107$.
- Por causa do fecho, L_5 é a única linguagem INFINITA do projeto; as outras quatro são finitas.
- Rejeita mensagem vazia, dois espaços seguidos e símbolos fora de Σ_5 (ex.: pressão, com ã).

Como garantimos que é a mesma linguagem

1. Código → AFNε (Thompson)

O AFNε é construído a partir do padrão exato do código.

2. ER formal $\equiv$ código: prova

Busca em largura sobre o produto dos dois AFNε (construção de subconjuntos). Nenhuma cadeia distingue as linguagens.

3. Arquivo .jff $\equiv$ código: prova

O .jff é lido de volta e comparado pelo mesmo algoritmo.

4. Executado no próprio JFLAP 7.1

80 de 80 cadeias de teste com o resultado esperado.

Verificação	Resultado
ER formal $\equiv$ código	5/5 provadas
.jff $\equiv$ código	5/5 provadas
AFNε × re (aleatórias)	10.000 cadeias, 0 divergências
JFLAP 7.1	80/80
Linguagens disjuntas	interseções vazias

Achado: o JFLAP 7.1 trata rótulos com '[' como intervalo ([a-z]) e falha com o símbolo '['. Por isso a severidade é INFO, não [INFO]: o .jff reconhece a linguagem exata, sem símbolos substitutos.

Demonstração

```
python src/auditor.py dados/turno_caldeira.txt          # lote + relatório + filtros
python src/auditor.py --pacote "CMD ATU-VLV-0201 AJUSTAR 120%"
python src/auditor.py --explicar "ATU-VLV-0420"         # simulação do AFNε
JFLAP: automatos/jflap/ER-03.jff → Input › Multiple Run → Load Inputs (entradas/ER-03.txt)
```

[ERRO] ER-03 Comando de Controle: valor de ajuste 120% acima de 100%; o intervalo aceito é 0 a 100%.
CMD ATU-VLV-0201 AJUSTAR 120%

^

Onde: coluna 28: símbolo '0' inesperado; esperado '%'

passo	lido	conjunto de estados ativos (após o fecho- ϵ)
0	–	{q0, q1, q2}
1	'A'	{q4}
2	'T'	{q6}
⋮		
12	'0'	{q18}

Resultado: ACEITA – o último conjunto contém o estado final (q18).

Testes e resultados

Arquivo	O que verifica	Testes
test_aplicacao.py	Entradas vazias/inválidas, diagnóstico, lote, CLI	66
test_automatos.py	AFN ϵ $\times$ re, provas de equivalência, jff, linguagens disjuntas	118
test_expressoes.py	Cadeias aceitas/rejeitadas, requisitos 6+6, categoria única	130
Total	python -m pytest	314 ✓

Casos-limite cobertos

- 0% e 100%; 850 e 1099 hPa
- octetos 0 e 255; máscaras /0 e /32
- 29/02, 30/04, 31/07 aceitas; 30/02, 31/04, hora 24 rejeitadas
- menor e maior cadeia de L_1 ; a cadeia vazia ϵ

Além das 80 cadeias de teste

- JFLAP 7.1: 80/80 com o resultado esperado
- 10.000 cadeias aleatórias: 0 divergências entre AFN ϵ e re
- Dados de exemplo: 20 íntegros e 14 corrompidos, todos com a causa correta

Limitações e possíveis melhorias

Limitações (falsos resultados)

- A validação é léxica: um ID bem formado passa mesmo se o dispositivo não existir.
- -0C é aceito; uma leitura real acima de 199.9 °C é bloqueada.
- 29/02 é aceito em qualquer ano.
- A mensagem do log não aceita acentos.

Melhorias

- Ano bissexto: também é regular (anos de 4 dígitos formam um conjunto finito), mas o AFNε cresceria muito.
- Faixas configuráveis por tipo de sensor.
- Leitura contínua de porta serial ou socket.
- Minimizar o AFD (Hopcroft) e comparar com o AFNε de Thompson.

Contribuições e uso de IA

Integrante	Contribuições
Augusto Rodrigues	ER-01 e ER-02 (fichas, AFNe, testes); diagnóstico de falhas
Caue Jadão	ER-03 e ER-04; revisão geral, verificação de equivalência, relatório e repositório
César Augusto	ER-05; modo lote, relatório de conformidade, exportação e apresentação

Uso de IA (declarado no relatório e no README)

- Antigravity: versão inicial do código, dos testes e dos primeiros .jff.
- Claude (Anthropic): revisão das ERs e da notação formal, construção de Thompson, verificação de equivalência, testes, relatório e slides.
- As decisões de projeto são da equipe; todos compreendem, explicam e modificam o conteúdo.

Obrigado!

Perguntas?

github.com/cauebarroso/Auditor-de-sensores-industriais-com-express-es-regulares